DUAL_GANTRY_GUIDE

듀얼 갠트리 모터 제어 가이드

ROS2 Humble / EtherCAT (pysoem) 기반 **2대 갠트리** 동시 제어 패키지.
단일 EtherCAT 체인에 6대 드라이버를 연결하고, 단일 ROS2 노드로 두 갠트리를 제어합니다.

목차

1. 시스템 구성
2. 빌드 및 실행
3. ROS2 인터페이스
4. 호밍
5. 이동 명령
6. 설정 파일
7. 속도 단위 변환
8. 동기화 (Cross Coupling)
9. 테스트 모드

---

1. 시스템 구성

```
[Jetson — Ubuntu 22.04]
        │ EtherCAT (enP8p1s0)
        ├── Slave 0 : iX7NH → 갠트리0 X축  (NOT 리밋)
        ├── Slave 1 : iX7NH → 갠트리0 Z1축 (POT 리밋)
        ├── Slave 2 : iX7NH → 갠트리0 Z2축 (POT 리밋)
        ├── Slave 3 : iX7NH → 갠트리1 X축  (NOT 리밋)
        ├── Slave 4 : iX7NH → 갠트리1 Z1축 (POT 리밋)
        └── Slave 5 : iX7NH → 갠트리1 Z2축 (POT 리밋)
```

갠트리	X 슬레이브	Z1 슬레이브	Z2 슬레이브
갠트리 0	0	1	2
갠트리 1	3	4	5

단일 노드 아키텍처: `motor_driver_node` 하나가 두 갠트리 모두를 관장합니다.
두 갠트리는 동일한 EtherCAT 체인에 직렬로 연결되어 있습니다.

---

2. 빌드 및 실행

```
# 빌드
cd /home/ihow/motor_ws
source /opt/ros/humble/setup.bash
colcon build --symlink-install

# 실행 (반드시 root — EtherCAT CAP_NET_RAW 필요)
sudo -s
source /opt/ros/humble/setup.bash
source /home/ihow/motor_ws/install/setup.bash
ros2 launch motor_control motor_control.launch.py
```

정상 시작 로그:

```
[EtherCAT] 6개 슬레이브 발견
  Slave 0 (X): 호밍 모드(6) 사전 설정 완료 (method=1, fast=30.0RPM, slow=1.0RPM)
  Slave 1 (Z): 호밍 모드(6) 사전 설정 완료 (method=2, fast=30.0RPM, slow=1.0RPM)
  Slave 2 (Z): 호밍 모드(6) 사전 설정 완료 (method=2, fast=30.0RPM, slow=1.0RPM)
  Slave 3 (X): 호밍 모드(6) 사전 설정 완료 (method=1, fast=30.0RPM, slow=1.0RPM)
  Slave 4 (Z): 호밍 모드(6) 사전 설정 완료 (method=2, fast=30.0RPM, slow=1.0RPM)
  Slave 5 (Z): 호밍 모드(6) 사전 설정 완료 (method=2, fast=30.0RPM, slow=1.0RPM)
[EtherCAT] OP 상태 진입 완료
motor_driver_node 준비 완료.
```

---

3. ROS2 인터페이스

토픽

토픽	타입	설명
`/motor/gantry0/axis_state`	`AxisState`	갠트리0 위치·속도·호밍 상태 (10Hz)
`/motor/gantry1/axis_state`	`AxisState`	갠트리1 위치·속도·호밍 상태 (10Hz)

AxisState 필드:

필드	타입	설명
`x_position_mm`	float64	X축 현재 위치 [mm]
`z_position_mm`	float64	Z축 평균 위치 [mm] (Z1·Z2 평균)
`z1_position_mm`	float64	Z1 모터 개별 위치 [mm]
`z2_position_mm`	float64	Z2 모터 개별 위치 [mm]
`x_velocity_mm_s`	float64	X축 현재 속도 [mm/s]
`z_velocity_mm_s`	float64	Z축 현재 속도 [mm/s]
`z_sync_error_mm`	float64	Z1-Z2 동기화 오차 [mm]
`x_homed`	bool	X축 원점 복귀 완료

필드	타입	설명
`z_homed`	bool	Z축 원점 복귀 완료
`drive_state`	string	CiA 402 상태 문자열
`sync_fault`	bool	동기화 오차 비상 정지 여부

```
ros2 topic echo /motor/gantry0/axis_state --once
ros2 topic echo /motor/gantry1/axis_state --once
```

서비스

`/motor/get_status`

타입: `motor_control_interfaces/srv/GetDriveStatus`

```
# 요청
int32 gantry_index   # 0=갠트리0, 1=갠트리1
---
# 응답
bool    ethercat_connected
string  cia402_state
bool    x_homed
bool    z_homed
float64 x_position_mm
float64 z_position_mm
string  error_code
bool    sync_fault
```

```
# 갠트리0 상태
ros2 service call /motor/get_status motor_control_interfaces/srv/GetDriveStatus \
  "{gantry_index: 0}"

# 갠트리1 상태
ros2 service call /motor/get_status motor_control_interfaces/srv/GetDriveStatus \
  "{gantry_index: 1}"
```

`/motor/set_velocity`

타입: `motor_control_interfaces/srv/SetVelocity`

두 갠트리에 **공통으로** 적용됩니다.

```
# 요청
float64 velocity_mm_s   # 목표 속도 [mm/s]
string  axis            # "x" | "z" | "all"
---
# 응답
bool   success
string message
```

```
ros2 service call /motor/set_velocity motor_control_interfaces/srv/SetVelocity \
  "{velocity_mm_s: 30.0, axis: 'all'}"
```

액션

`/motor/homing`

타입: `motor_control_interfaces/action/Homing`

```
# Goal
int32 gantry_index   # 0=갠트리0, 1=갠트리1, 2=동시 호밍
bool  force_rehome   # 이미 완료 상태여도 재실행 여부
---
# Result
bool    success
string  message
float64 gantry0_z_home_position_mm
float64 gantry1_z_home_position_mm
---
# Feedback
string  phase                   # "in_progress" | "idle"
float64 gantry0_current_z_mm
float64 gantry1_current_z_mm
```

`/motor/move`

타입: `motor_control_interfaces/action/MoveAxis`

```
# Goal
int32   gantry_index            # 0=갠트리0, 1=갠트리1, 2=동시 이동

float64 gantry0_x_target_mm    # 갠트리0 X 목표 [mm]  (-1.0e9 = 현 위치 유지)
float64 gantry0_z_target_mm    # 갠트리0 Z 목표 [mm]
float64 gantry1_x_target_mm    # 갠트리1 X 목표 [mm]
float64 gantry1_z_target_mm    # 갠트리1 Z 목표 [mm]
float64 gantry2_x_target_mm    # 동시 이동 공통 X 목표 [mm]  (gantry_index=2 시 사용)
float64 gantry2_z_target_mm    # 동시 이동 공통 Z 목표 [mm]

float64 velocity_mm_s           # 이동 속도 [mm/s] (0 = 기본값 사용)
bool    force_move              # true = 호밍 미완료·소프트 리밋 무시
---
# Result
bool    success
float64 gantry0_x_final_mm
float64 gantry0_z_final_mm
float64 gantry1_x_final_mm
float64 gantry1_z_final_mm
string  message
---
# Feedback
float64 gantry0_x_current_mm
```

```
float64 gantry0_z_current_mm
float64 gantry1_x_current_mm
float64 gantry1_z_current_mm
float64 gantry0_x_remaining_mm
float64 gantry0_z_remaining_mm
float64 gantry1_x_remaining_mm
float64 gantry1_z_remaining_mm
```

> **규칙:** 이동하지 않을 축에는 반드시 `-1.0e9` 를 지정합니다.
> `gantry_index=0` 이면 `gantry1_*` 필드는 무시됩니다. `gantry_index=2` 이면 `gantry2_*` 필드를 사용합니다.

---

4. 호밍

호밍 방법

축	Method	동작
X	1 (NOT + Index)	역방향(-) → NOT 리밋 → Index 펄스에서 원점
Z	2 (POT + Index)	정방향(+) → POT 리밋 → Index 펄스에서 원점

gantry_index 별 동작

gantry_index	호밍 대상
`0`	갠트리0 (Slave 0·1·2)
`1`	갠트리1 (Slave 3·4·5)
`2`	갠트리0 + 갠트리1 동시

호밍 명령 예시

```
# 두 갠트리 동시 호밍 (권장)
ros2 action send_goal /motor/homing motor_control_interfaces/action/Homing \
  "{gantry_index: 2, force_rehome: true}"

# 갠트리0만 호밍
ros2 action send_goal /motor/homing motor_control_interfaces/action/Homing \
  "{gantry_index: 0, force_rehome: true}"

# 갠트리1만 호밍
ros2 action send_goal /motor/homing motor_control_interfaces/action/Homing \
  "{gantry_index: 1, force_rehome: true}"

# 이미 호밍된 경우 스킵
ros2 action send_goal /motor/homing motor_control_interfaces/action/Homing \
  "{gantry_index: 2, force_rehome: false}"
```

5. 이동 명령

5-1. 두 갠트리 동시·동일 이동 (gantry_index=2)

`gantry2_x_target_mm`, `gantry2_z_target_mm` 에 공통 목표를 지정합니다.
나머지 `gantry0_*`, `gantry1_*` 필드는 모두 `-1.0e9` 로 설정합니다.

```
# 두 갠트리 동시 X=100mm, Z=-50mm 이동
ros2 action send_goal /motor/move motor_control_interfaces/action/MoveAxis "{
  gantry_index: 2,
  gantry0_x_target_mm: -1.0e9, gantry0_z_target_mm: -1.0e9,
  gantry1_x_target_mm: -1.0e9, gantry1_z_target_mm: -1.0e9,
  gantry2_x_target_mm: 100.0,  gantry2_z_target_mm: -50.0,
  velocity_mm_s: 30.0, force_move: false}"

# 두 갠트리 동시 X축만 이동 (Z 유지)
ros2 action send_goal /motor/move motor_control_interfaces/action/MoveAxis "{
  gantry_index: 2,
  gantry0_x_target_mm: -1.0e9, gantry0_z_target_mm: -1.0e9,
  gantry1_x_target_mm: -1.0e9, gantry1_z_target_mm: -1.0e9,
  gantry2_x_target_mm: 500.0,  gantry2_z_target_mm: -1.0e9,
  velocity_mm_s: 75.0, force_move: false}"

# 두 갠트리 동시 원점 복귀
ros2 action send_goal /motor/move motor_control_interfaces/action/MoveAxis "{
  gantry_index: 2,
  gantry0_x_target_mm: -1.0e9, gantry0_z_target_mm: -1.0e9,
  gantry1_x_target_mm: -1.0e9, gantry1_z_target_mm: -1.0e9,
  gantry2_x_target_mm: 0.0,    gantry2_z_target_mm: 0.0,
  velocity_mm_s: 75.0, force_move: false}"
```

5-2. 갠트리0만 이동 (gantry_index=0)

`gantry0_x_target_mm`, `gantry0_z_target_mm` 을 지정합니다.
나머지 필드는 모두 `-1.0e9` 로 설정합니다.

```
# 갠트리0: X=200mm, Z=-80mm
ros2 action send_goal /motor/move motor_control_interfaces/action/MoveAxis "{
  gantry_index: 0,
  gantry0_x_target_mm: 200.0,  gantry0_z_target_mm: -80.0,
  gantry1_x_target_mm: -1.0e9, gantry1_z_target_mm: -1.0e9,
  gantry2_x_target_mm: -1.0e9, gantry2_z_target_mm: -1.0e9,
  velocity_mm_s: 30.0, force_move: false}"

# 갠트리0: X축만 이동
ros2 action send_goal /motor/move motor_control_interfaces/action/MoveAxis "{
  gantry_index: 0,
  gantry0_x_target_mm: 500.0,  gantry0_z_target_mm: -1.0e9,
  gantry1_x_target_mm: -1.0e9, gantry1_z_target_mm: -1.0e9,
```

```
  gantry2_x_target_mm: -1.0e9, gantry2_z_target_mm: -1.0e9,
  velocity_mm_s: 50.0, force_move: false}"
```

5-3. 갠트리1만 이동 (gantry_index=1)

```
# 갠트리1: X=150mm, Z=-100mm
ros2 action send_goal /motor/move motor_control_interfaces/action/MoveAxis "{
  gantry_index: 1,
  gantry0_x_target_mm: -1.0e9, gantry0_z_target_mm: -1.0e9,
  gantry1_x_target_mm: 150.0,  gantry1_z_target_mm: -100.0,
  gantry2_x_target_mm: -1.0e9, gantry2_z_target_mm: -1.0e9,
  velocity_mm_s: 30.0, force_move: false}"

# 갠트리1: Z축만 이동
ros2 action send_goal /motor/move motor_control_interfaces/action/MoveAxis "{
  gantry_index: 1,
  gantry0_x_target_mm: -1.0e9, gantry0_z_target_mm: -1.0e9,
  gantry1_x_target_mm: -1.0e9, gantry1_z_target_mm: -200.0,
  gantry2_x_target_mm: -1.0e9, gantry2_z_target_mm: -1.0e9,
  velocity_mm_s: 20.0, force_move: false}"
```

5-4. 강제 이동 (호밍 미완료, 소프트 리밋 무시)

```
ros2 action send_goal /motor/move motor_control_interfaces/action/MoveAxis "{
  gantry_index: 0,
  gantry0_x_target_mm: 0.0,    gantry0_z_target_mm: -30.0,
  gantry1_x_target_mm: -1.0e9, gantry1_z_target_mm: -1.0e9,
  gantry2_x_target_mm: -1.0e9, gantry2_z_target_mm: -1.0e9,
  velocity_mm_s: 10.0, force_move: true}"
```

---

6. 설정 파일 (motor_params.yaml)

경로: `src/motor_control/config/motor_params.yaml`

전체 파라미터

파라미터	기본값	설명
`ifname`	`enP8p1s0`	EtherCAT 인터페이스 이름
`num_slaves`	`6`	총 슬레이브 수
`pdo_cycle_ms`	`10`	PDO 사이클 [ms]
`state_publish_hz`	`10.0`	AxisState 발행 주기 [Hz]
`gantry0_x_slave_idx`	`0`	갠트리0 X축 슬레이브 인덱스
`gantry0_z1_slave_idx`	`1`	갠트리0 Z1축 슬레이브 인덱스
`gantry0_z2_slave_idx`	`2`	갠트리0 Z2축 슬레이브 인덱스

파라미터	기본값	설명
`gantry1_x_slave_idx`	`3`	갠트리1 X축 슬레이브 인덱스
`gantry1_z1_slave_idx`	`4`	갠트리1 Z1축 슬레이브 인덱스
`gantry1_z2_slave_idx`	`5`	갠트리1 Z2축 슬레이브 인덱스
`default_velocity_rpm`	`30.0`	기본 속도 [RPM]
`default_velocity_mm_s`	`75.0`	기본 속도 [mm/s]
`default_accel_rpm_s`	`1000.0`	기본 가감속 [RPM/s]
`x_homing_method`	`1`	X축 호밍 메서드
`z_homing_method`	`2`	Z축 호밍 메서드
`homing_fast_rpm`	`30.0`	호밍 탐색 속도 [RPM]
`homing_slow_rpm`	`1.0`	호밍 Index 탐색 속도 [RPM]
`homing_timeout_s`	`1200.0`	호밍 타임아웃 [초]
`enable_coupling`	`true`	Z1-Z2 Cross-Coupling 활성화
`coupling_gain`	`0.05`	Cross-Coupling 게인 [1/s]
`max_sync_error_mm`	`10.0`	동기화 오차 긴급정지 임계값 [mm]
`ma_window`	`5`	동기화 오차 이동평균 윈도우
`gantry0_x_soft_limit_min_mm`	`0.0`	갠트리0 X 소프트 리밋 하한 [mm]
`gantry0_x_soft_limit_max_mm`	`1750.0`	갠트리0 X 소프트 리밋 상한 [mm]
`gantry0_z_soft_limit_min_mm`	`-1750.0`	갠트리0 Z 소프트 리밋 하한 [mm]
`gantry0_z_soft_limit_max_mm`	`0.1`	갠트리0 Z 소프트 리밋 상한 [mm]
`gantry1_x_soft_limit_min_mm`	`0.0`	갠트리1 X 소프트 리밋 하한 [mm]
`gantry1_x_soft_limit_max_mm`	`1750.0`	갠트리1 X 소프트 리밋 상한 [mm]
`gantry1_z_soft_limit_min_mm`	`-1750.0`	갠트리1 Z 소프트 리밋 하한 [mm]
`gantry1_z_soft_limit_max_mm`	`0.1`	갠트리1 Z 소프트 리밋 상한 [mm]

---

7. 속도 단위 변환

```
X축:  RPM = mm/s × 60 ÷ 11.999
Z축:  RPM = mm/s × 60 ÷ 5.999

예시) X축 50 mm/s → 50 × 60 ÷ 11.999 ≈ 250 RPM
      Z축 20 mm/s → 20 × 60 ÷  5.999 ≈ 200 RPM
```

자세한 내용: [VELOCITY_GUIDE.md]

---

8. 동기화 (Cross Coupling)

각 갠트리의 Z1↔Z2 두 모터는 **갠트리별로 독립적으로** 동기화 모니터링 및 Cross Coupling이 적용됩니다.

- `max_sync_error_mm` 초과 시 해당 갠트리의 모든 모터 긴급정지
- `coupling_gain` 으로 보정 강도 조절
- `enable_coupling: false` 로 비활성화 가능
- `sync_fault` 플래그로 상태 확인: `ros2 topic echo /motor/gantry0/axis_state --once`

---

9. 테스트 모드

사용하지 않는 슬레이브는 `-1` 로 설정하고 `num_slaves` 를 실제 연결 대수로 조정합니다.

갠트리0 X축 단독

```
num_slaves: 1
gantry0_x_slave_idx:  0
gantry0_z1_slave_idx: -1
gantry0_z2_slave_idx: -1
gantry1_x_slave_idx:  -1
gantry1_z1_slave_idx: -1
gantry1_z2_slave_idx: -1
```

갠트리0 전체 (X+Z1+Z2)

```
num_slaves: 3
gantry0_x_slave_idx:  0
gantry0_z1_slave_idx: 1
gantry0_z2_slave_idx: 2
gantry1_x_slave_idx:  -1
gantry1_z1_slave_idx: -1
gantry1_z2_slave_idx: -1
```

갠트리1 전체 (갠트리0 없이)

```
num_slaves: 3
gantry0_x_slave_idx:  -1
gantry0_z1_slave_idx: -1
gantry0_z2_slave_idx: -1
gantry1_x_slave_idx:  0
gantry1_z1_slave_idx: 1
gantry1_z2_slave_idx: 2
```

듀얼 갠트리 전체

```
num_slaves: 6
gantry0_x_slave_idx:  0
gantry0_z1_slave_idx: 1
gantry0_z2_slave_idx: 2
```

```yaml
gantry1_x_slave_idx:  3
gantry1_z1_slave_idx: 4
gantry1_z2_slave_idx: 5
```